SSL & TLS
Module 4

Prepared by

Jesna J S

Assistant Professor, CSE TKMCE

Topics Covered

- Introduction to web security
  - Web security considerations
  - Threats.
- Secure Sockets Layer (SSL)
  - Architecture
  - Protocols
- Transport Layer Security (TLS)
  - Differences from SSL.

Web Security

- The World Wide Web is fundamentally a client/server application running over the Internet and TCP/IP intranets.

Challenges that the web poses

1. Vulnerability to Attacks: Web servers can be targeted over the Internet.
2. Reputation & Financial Loss: A compromised server can damage trust and lead to financial losses.
3. Complex Software Risks: Easy to configure, but complex software may hide security flaws.
4. Gateway to Other Systems: Attackers may access sensitive data beyond the web server.
5. Lack of User Awareness: Untrained users may not recognize security threats or know how to prevent them.

Web Security Threats

	Threats	Consequences	Countermeasures
Integrity	• Modification of user data • Trojan horse browser • Modification of memory • Modification of message traffic in transit	• Loss of information • Compromise of machine • Vulnerabilty to all other threats	Cryptographic checksums
Confidentiality	• Eavesdropping on the net • Theft of info from server • Theft of data from client • Info about network configuration • Info about which client talks to server	• Loss of information • Loss of privacy	Encryption, Web proxies
Denial of Service	• Killing of user threads • Flooding machine with bogus requests • Filling up disk or memory • Isolating machine by DNS attacks	• Disruptive • Annoying • Prevent user from getting work done	Difficult to prevent
Authentication	• Impersonation of legitimate users • Data forgery	• Misrepresentation of user • Belief that false information is valid	Cryptographic techniques

Secure Socket Layer & Transport Layer Security (SSL & TLS)

- **Transport layer security provides end-to-end security service for application that use a reliable transport layer protocol such as TCP.**
- The idea is to provide security service for transactions on the internet.
- Eg When a customer shops online the following security service desired.
  1. The customer needs to be sure that the server belongs to the actual vendor, not an impostor.
  2. The customer and the vendor need to be sure that the contents of the message are not modified during transmission, message integrity.
  3. An imposter does not intercept sensitive information such as a credit card number, confidentiality.

- Two protocols are dominated today for providing security at the transport layer:
  1. **Secure Socket Layer, SSL**
  2. **Transport Layer Security, TLS**
- One of the goals of the protocol is to provide server and client authentication, data confidentiality and data integrity.
- Application layer client-server programs such HTTP that use the service of TCP, encapsulate their data in SSL packet.

SSL(Secure Socket Layer)

- **SSL is designed to provide security and compression service to data generated from an application layer.**
- Typically SSL can receive data from any application layer protocol(HTTP) which is compressed (optional), signed ,and encrypted.
- The data is then passed to a reliable transport layer protocol such as TCP.
- NETSCAPE developed SSL in 1994 .Versions 2 & 3 were released in 1995.

1. SERVICES

SSL provides several services on data received from the application layer.

1. Fragmentation: First SSL divides the data into blocks of 2^{14} bytes or less.
2. Compression: Each fragments of data is compressed using one of the lossless compression methods between the client and server. (Optional)
3. Message Integrity: To preserve integrity of data, SSL uses a keyed-hash function to create a MAC.
4. Confidentiality: To provide confidentiality, the original data and MAC are encrypted using symmetric-key cryptography.
5. Framing: A header is added to the encrypted payload. The payload is then passed to a reliable transport layer protocol.

SSL ARCHITECTURE

- SSL is built on TCP to provide a secure and reliable communication service.
- It consists of two layers of protocols:
  a) SSL Record Protocol – Provides basic security services.
  b) Higher-Layer Protocols – Includes:
     1. Handshake Protocol – Establishes security parameters.
     2. Change Cipher Spec Protocol – Updates encryption settings.
     3. Alert Protocol – Manages error messages.
- Two important SSL concepts are the SSL session and the SSL connection

SSL ARCHITECTURE

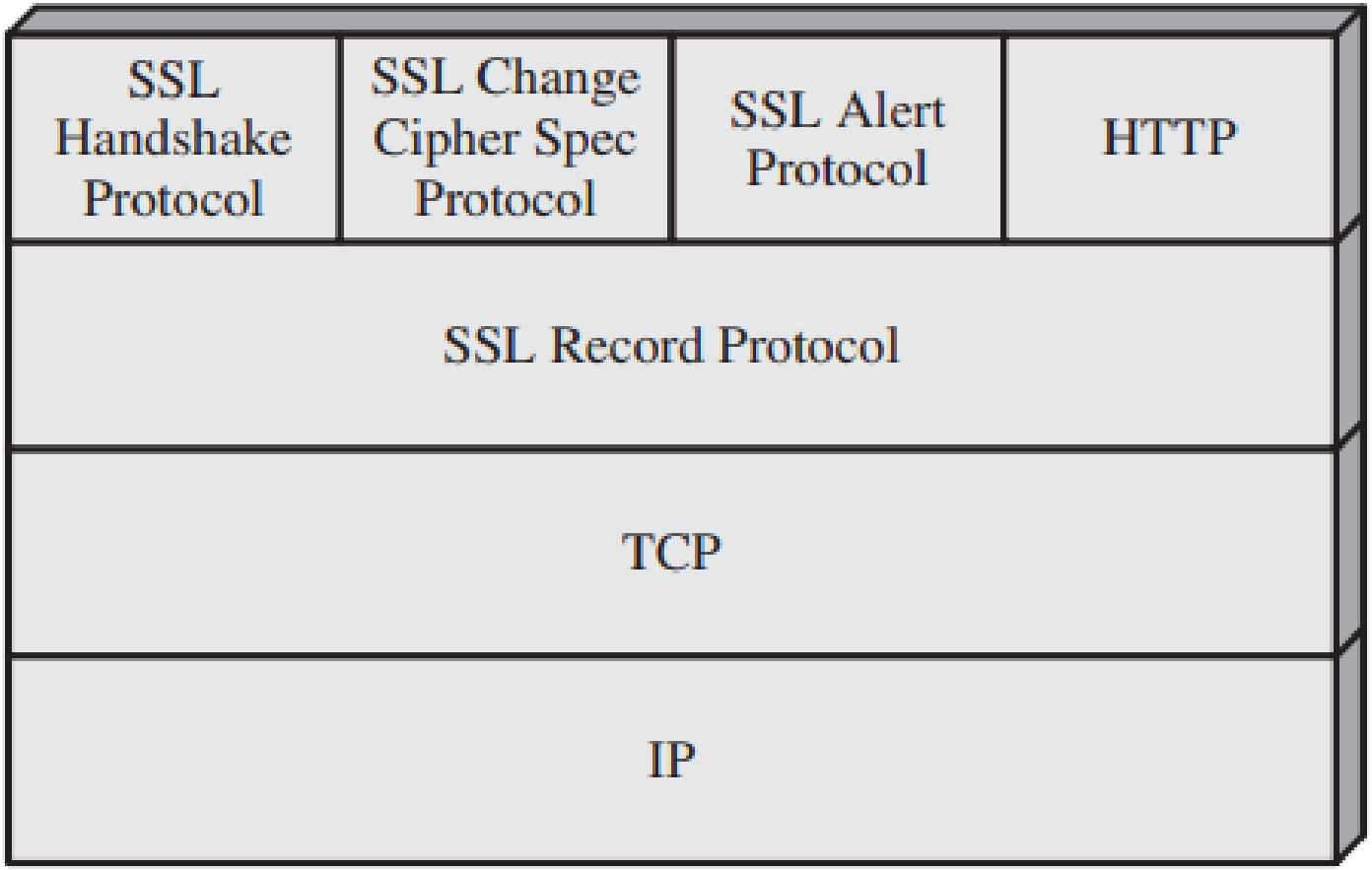


Figure 16.2 SSL Protocol Stack

SSL Connection

- A connection is a transport that provides a suitable type of service
- Connections holds peer-to-peer relationships and transient in nature.
- **It creates temporary link between two parties** (client & server).
- **Every connection belongs to one session**

SSL Connection Parameters

1. **Server & Client Random:** Random byte sequences for each connection.
2. **Write MAC Secrets:** Separate keys for client & server message authentication.
3. **Write Keys:** Encryption keys for securing data transmission.
4. **Initialization Vectors (IV):** Used in block cipher encryption (CBC mode).
5. **Sequence Numbers:**
   - Each party maintains sequence numbers for messages.
   - Reset to zero when encryption settings change and cannot exceed $2^{64} - 1$.

SSL Session

- **A long-term association between a client and a server.**
- Sessions are created by the **Handshake Protocol**.
- Defined by cryptographic security parameters which can be shared among multiple connections.
- Allows multiple connections without re-negotiating security settings.

SSL Session Parameters

1. **Session ID:** Identifies an active or resumable session.
2. **Peer Certificate:** Optional X.509 certificate for authentication.
3. **Compression Method:** Algorithm used to compress data before encryption.
4. **Cipher Spec:** Defines encryption and hashing algorithms (e.g., AES, SHA-1).
5. **Master Secret:** A **48-byte shared secret** between client and server.
6. **Resumable Flag:** Indicates if the session can be reused for new connections.

SSL RECORD PROTOCOL

- It operates at the **transport layer** and is responsible for ensuring **secure communication** between a client (e.g., web browser) and a server (e.g., website).
- The SSL Record Protocol provides two services for SSL connections:
  1. Confidentiality: The Handshake Protocol defines a shared secret key that is used for conventional encryption of SSL payloads. •
  2. Message Integrity: The Handshake Protocol also defines a shared secret key that is used to form a message authentication code (MAC)

SSL Record Protocol Operation

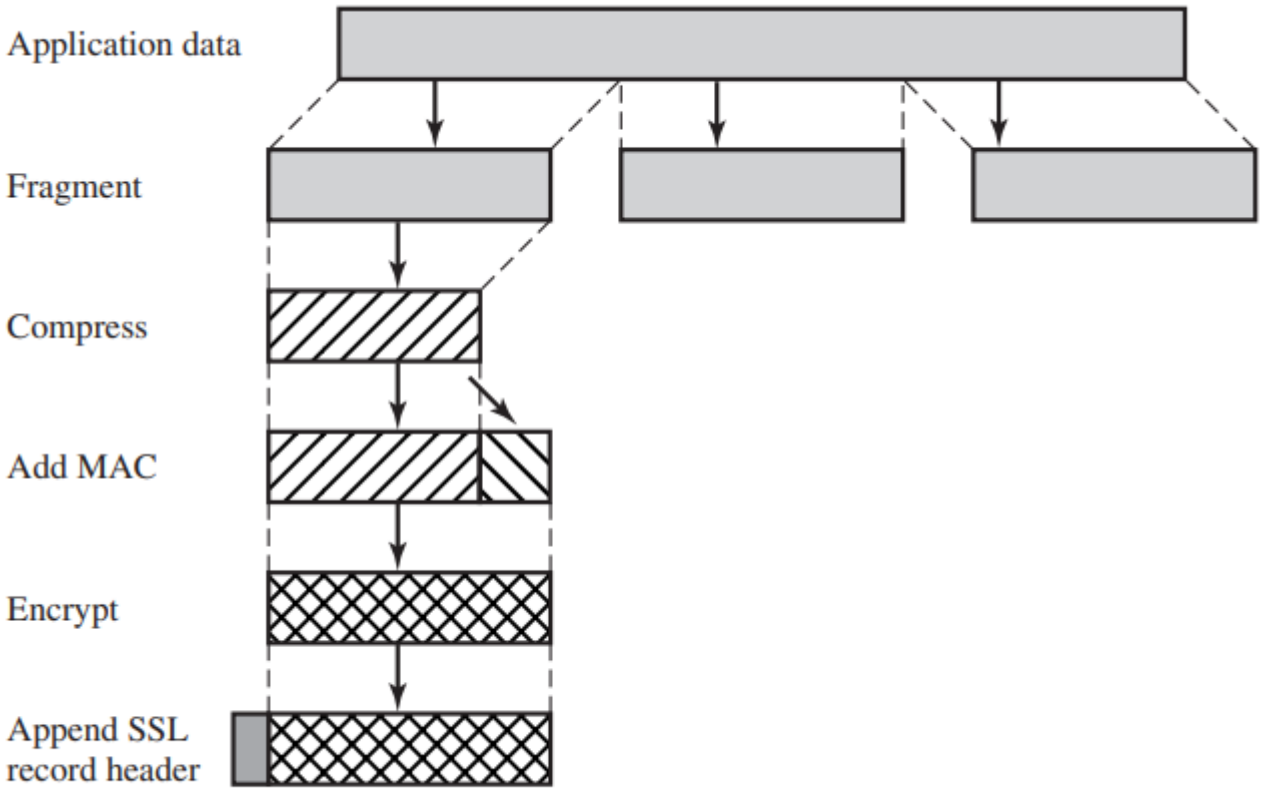


Figure 16.3 SSL Record Protocol Operation

1. The Record Protocol takes an application message to be transmitted,
2. Fragments the data into manageable blocks
3. Optionally compresses the data
4. Applies a MAC
5. Encrypts
6. Adds a header, and transmits the resulting unit in a TCP segment.
7. Received data are decrypted, verified, decompressed, and reassembled before being delivered to higher-level users.

SSL Record Header Format

- The header consisting of the following fields:
- Content Type (8 bits): The higher-layer protocol used to process the enclosed fragment.
- Major Version (8 bits): Indicates major version of SSL in use. For SSLv3, the value is 3.
- Minor Version (8 bits): Indicates minor version in use. For SSLv3, the value is 0.
- Compressed Length (16 bits): The length in bytes of the plaintext fragment (or compressed fragment if compression is used).

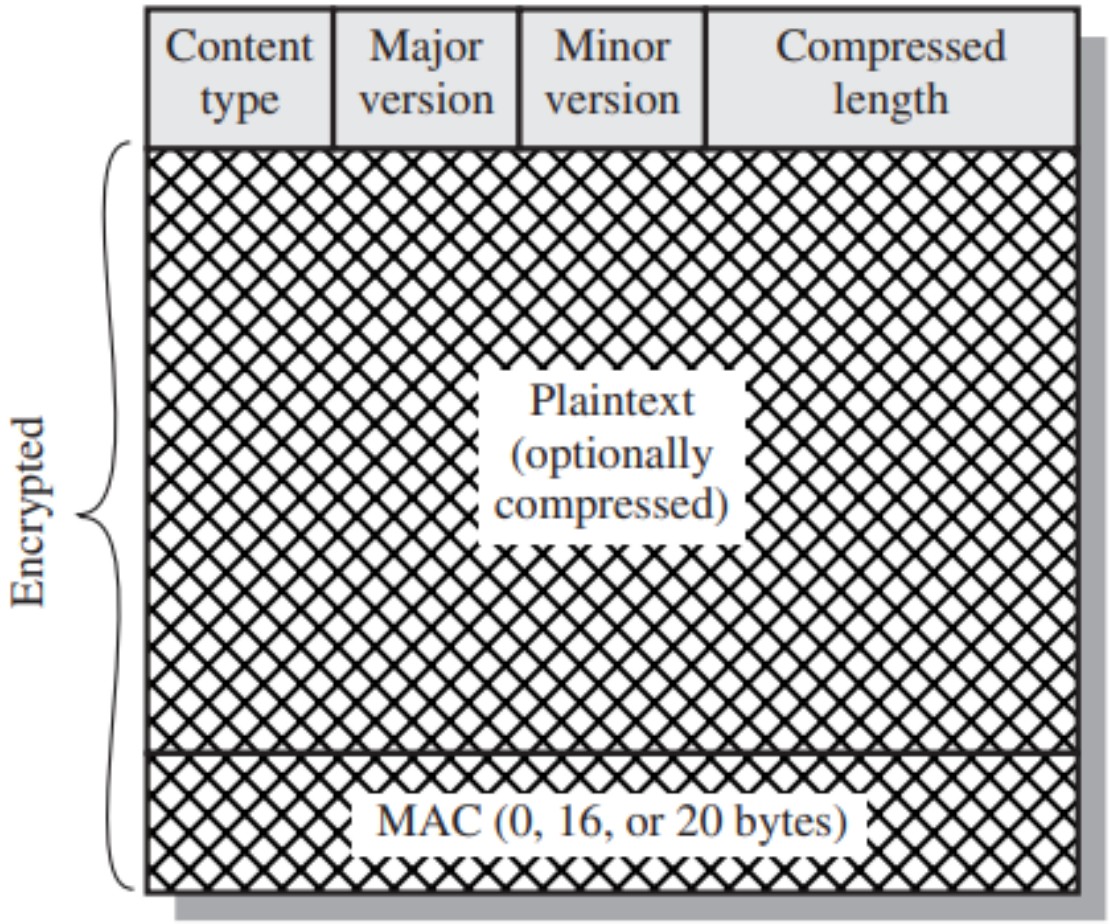


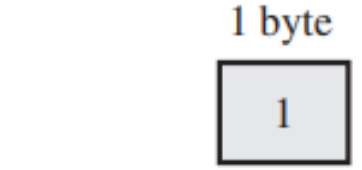
Figure 16.4 SSL Record Format

SSL Record Protocol Payload

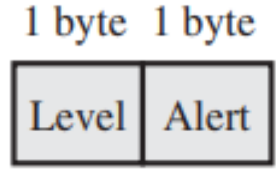
Payload = actual data carried inside the SSL Record

- It depends on what higher-level protocol is being protected.
- SSL Record can carry 4 types of content:

1. Change CipherSpec: switch to encryption
2. Alert Protocol :error/warning messages
3. Handshake Protocol: SSL/TLS handshake messages
4. Application Data: HTTP, FTP, SMTP etc.

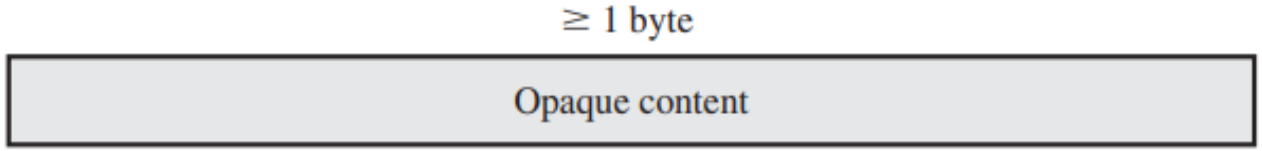


(a) Change Cipher Spec Protocol



(b) Alert Protocol

(c) Handshake Protocol



(d) Other Upper-Layer Protocol (e.g., HTTP)

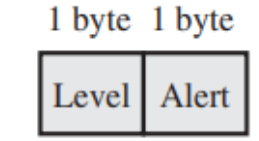
- SSL/TLS consists of three main subprotocols that work together to establish and maintain secure communication:
  1. Change Cipher spec protocol
  2. Handshake protocol
  3. Alert protocol

1. Change Cipher Spec Protocol

- The Change Cipher Spec Protocol is a small but crucial part of the SSL/TLS handshake.
- **It signals a transition from unencrypted to encrypted communication between the client and the server.**
- It **indicates** that both parties should now use the agreed-upon encryption and authentication settings established during the handshake.
- **Change Cipher Spec is a protocol-level message (single byte) signaling encryption activation**
- It contains **only a single byte** **(0x01)** and is sent right **before** the client and server begin using encryption
- **Finished Message** is an **encrypted handshake message** verifying that the handshake was completed successfully.

2. Alert Protocol

- The **Alert Protocol** is used for **error reporting and connection management** between the **client** and **server**.
- It ensures that both parties can handle issues like **invalid certificates, handshake failures, or security risks**.
- Each message in this protocol consists of two bytes.
- The first byte takes the value warning (1) or fatal (2) to convey the severity of the message.
- ***If the level is fatal, SSL immediately terminates the connection.***
- Other connections on the same session may continue, but no new connections on this session may be established.
- ***The second byte contains a code that indicates the specific alert***



(b) Alert Protocol

Alert types:

- **`unexpected_message:`** An inappropriate message was received.
- **`bad_record_mac:`** An incorrect MAC was received.
- **`decompression_failure:`** The decompression function received improper input (e.g., unable to decompress or decompress to greater than maximum allowable length).
- **`handshake_failure:`** Sender was unable to negotiate an acceptable set of security parameters given the options available.
- **`illegal_parameter:`** A field in a handshake message was out of range or inconsistent with other fields.

The remaining alerts are the following.

- **`close_notify:`** Notifies the recipient that the sender will not send any more messages on this connection. Each party is required to send a **`close_notify`** alert before closing the write side of a connection.

- **`no_certificate:`** May be sent in response to a certificate request if no appropriate certificate is available.
- **`bad_certificate:`** A received certificate was corrupt (e.g., contained a signature that did not verify).
- **`unsupported_certificate:`** The type of the received certificate is not supported.
- **`certificate_revoked:`** A certificate has been revoked by its signer.
- **`certificate_expired:`** A certificate has expired.

- **`certificate_unknown:`** Some other unspecified issue arose in processing the certificate, rendering it unacceptable.

3. Handshake Protocol

- The **SSL/TLS Handshake Protocol** is responsible for **establishing a secure communication channel** between a client (e.g., a web browser) and a server (e.g., a website).
- **It is the first step in an SSL/TLS connection, where encryption algorithms are negotiated, authentication is performed, and session keys are generated.**
- The Handshake Protocol consists of a series of messages exchanged by client and server.
- Each message has three fields:
  - ✓Type (1 byte): Indicates one of 10 messages.
  - ✓Length (3 bytes): The length of the message in bytes.
  - ✓Content (bytes): The parameters associated with this message

(c) Handshake Protocol

Table 16.2 SSL Handshake Protocol Message Types

Message Type	Parameters
`hello_request`	null
`client_hello`	version, random, session id, cipher suite, compression method
`server_hello`	version, random, session id, cipher suite, compression method
`certificate`	chain of X.509v3 certificates
`server_key_exchange`	parameters, signature
`certificate_request`	type, authorities
`server_done`	null
`certificate_verify`	signature
`client_key_exchange`	parameters, signature
`finished`	hash value

Handshake Protocol Action

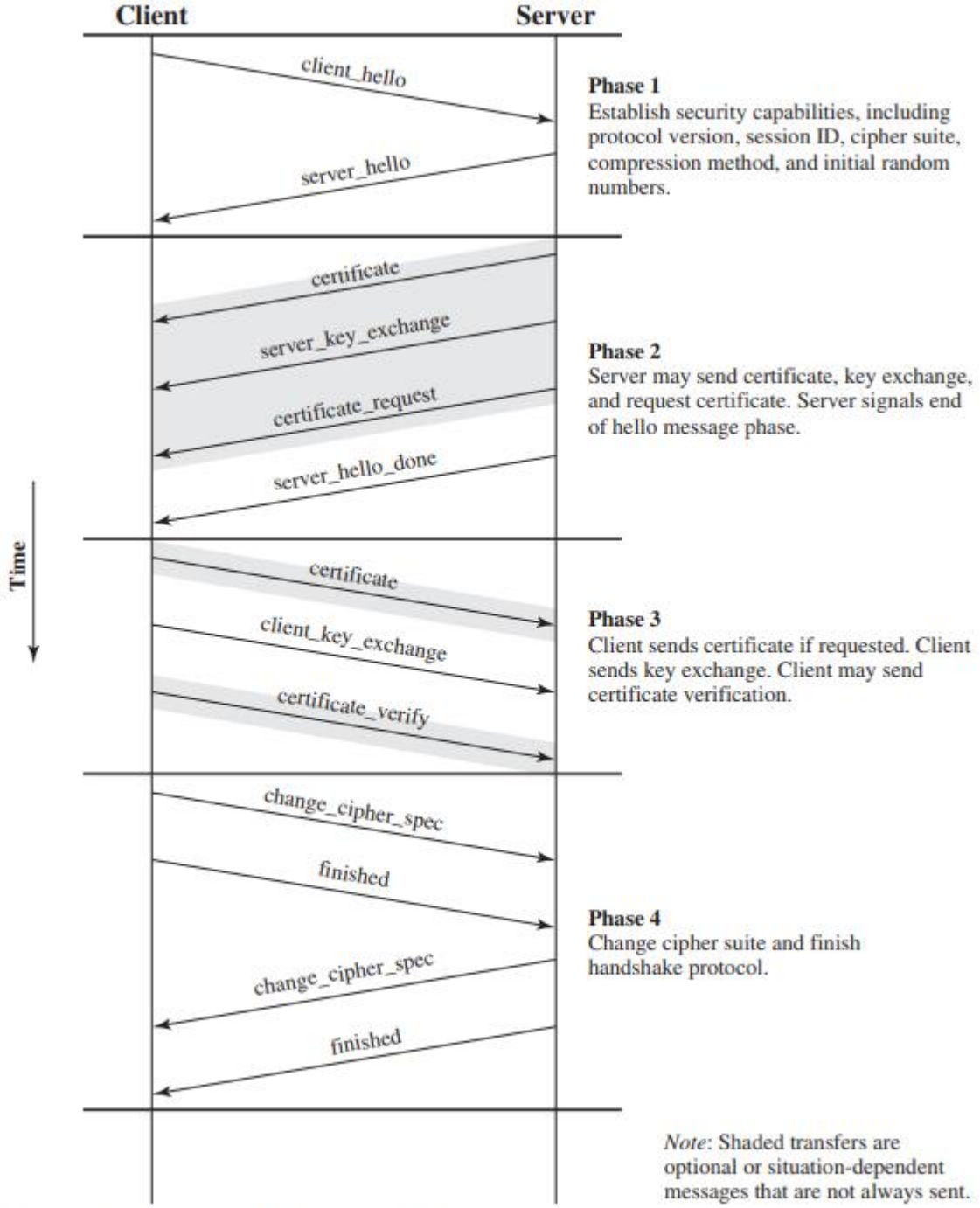


Figure 16.6 Handshake Protocol Action

Four phases of SSL

- The **SSL Handshake** is divided into **four phases**, as shown in the diagram.
- It ensures **secure communication** between a **client** (e.g., a web browser) and a **server** (e.g., a website).

1. PHASE 1: ESTABLISH SECURITY CAPABILITIES
2. PHASE 2: SERVER AUTHENTICATION AND KEY EXCHANGE
3. PHASE 3. CLIENT AUTHENTICATION AND KEY EXCHANGE
4. PHASE 4: FINISH

PHASE 1: ESTABLISH SECURITY CAPABILITIES

- This phase is used to **initiate a logical connection and to establish the security capabilities** that will be associated with it.
- The exchange is initiated by the client, which sends a client_hello message with the **following parameters**:

✓**Version** : Highest SSL version the client understands.

✓**Random** : A 32-bit timestamp + 28 random bytes (used to prevent replay attacks).

✓**Session ID** :

- **Nonzero** → Update existing session or create a new connection in the same session.
- **Zero** → Request a new session.

✓**CipherSuite** : List of supported cryptographic algorithms (in order of preference).

✓**Compression Method**: List of supported compression methods.

- After sending the client_hello message, the client waits for the server_hello message, which contains the same parameters as the client_hello message.
- **For the server_hello message, the following conventions apply.**
  - ✓Version field- ***lower of the client's suggested version*** and the server's highest supported version.
  - ✓Random field - ***Independent of the client's field***.
  - ✓If the client's ***SessionID was nonzero, the server uses the same value***. Otherwise, the server assigns a new one.
  - ✓CipherSuite - ***single cipher suite chosen by the server*** from the client's proposals.
  - ✓Compression- ***compression method chosen by the server*** from the client's proposals.

Key Exchange Methods in CipherSuite

1. **RSA**:
   - ✓Secret key is encrypted with the receiver's RSA public key.
   - ✓Requires a public key certificate for the receiver.
2. **Fixed Diffie-Hellman**:
   - ✓Server's certificate contains signed Diffie-Hellman public parameters.
   - ✓Client provides public parameters via a certificate or key exchange message.
   - ✓Results in a fixed secret key between peers.
3. **Ephemeral Diffie-Hellman**:
   - ✓Generates temporary (one-time) secret keys.
   - ✓Public keys are signed with RSA or DSS private key.
   - ✓Most secure Diffie-Hellman option due to authentication.

4. **Anonymous Diffie-Hellman**:
   - ✓No authentication; both sides exchange public parameters.
   - ✓Vulnerable to man-in-the-middle attacks.
5. **Fortezza**:
   - ✓Uses the Fortezza encryption scheme.
   - ✓ It was mainly implemented in U.S. government-approved secure web applications.

Phase 2: Server Authentication and Key Exchange

- The messages sent on this phase includes :

1. **Server Certificate Message** (if authentication is needed)
   - Contains one or a chain of X.509 certificates.
   - Required for all key exchange methods except anonymous Diffie-Hellman.
2. **Server Key Exchange Message** (sent if needed)
   - Not required if:
     1. The server's certificate already contains fixed Diffie-Hellman parameters.
     2. RSA key exchange is used (without a signature-only RSA key).

- Required for:
    a. Anonymous Diffie-Hellman → Contains Diffie-Hellman values + server's public key.
    b. Ephemeral Diffie-Hellman → Includes Diffie-Hellman parameters + signature.
    c. RSA with signature-only RSA key → Includes temporary RSA public key + signature.
    d. Fortezza → Uses the Fortezza scheme.

A signature is created by taking the hash of a message and encrypting it with the sender's private key.

- Hash is defined as:

```
hash(ClientHello.random || ServerHello.random ||
ServerParams)
```

(*all three values are concatenated together before hashing*)

- So, the hash covers not only the Diffie-Hellman or RSA parameters but also the two nonces from the initial hello messages. This ensures against replay attacks and misrepresentation.

3. Client Certificate Request (Optional)

- Non-anonymous servers can request a certificate from the client.
- Message Parameters:
    - Certificate Type:
      Defines public-key algorithm use.
    - Options:
        - ✓RSA (signature only, fixed Diffie-Hellman, ephemeral Diffie-Hellman).
        - ✓DSS (signature only, fixed Diffie-Hellman, ephemeral Diffie-Hellman).
        - ✓Fortezza.Certificate Authorities: List of acceptable certificate authority names.

4. Server Done Message (Always Required)

- Indicates the end of the server hello phase.
- Has no parameters.
- Server waits for client response

PHASE 3. CLIENT AUTHENTICATION AND KEY EXCHANGE

Upon receipt of the server_done message, the client should verify that the server provided a valid certificate (if required) and check that the server_hello parameters are acceptable. If all is satisfactory, the client sends one or more messages back to the server.

1. Client Certificate (If Requested by Server)

- Sent only if the server requested a certificate.
- If no valid certificate is available, the client sends a no_certificate alert.

2. Client Key Exchange Message (Always Required)

- Content depends on the key exchange method:
  - ✓RSA: Sends a 48-byte pre-master secret, encrypted with the server's public RSA key.
  - ✓Ephemeral/Anonymous Diffie-Hellman: Sends the client's Diffie-Hellman public parameters.

- ✓Fixed Diffie-Hellman: No additional message (parameters were in the certificate).
- ✓Fortezza: Sends the client's Fortezza parameters.

3. Certificate Verify Message (Optional)

- Sent only if the client's certificate has a signing capability.
- Provides explicit proof that the client owns the private key.
- Signs a hash of handshake messages using the client's private key:
  - ➢DSS key → Encrypts a SHA-1 hash.
  - ➢RSA key → Encrypts a concatenation of MD5 and SHA-1 hashes.
- ***Prevents unauthorized users from misusing the client certificate.***

Phase 4: Finish

1. **Finalizes Cipher Settings**
   - ✓Sends a **change_cipher_spec** message.
   - ✓Copies the **pending CipherSpec** into the **current CipherSpec**.
   - ✓This message is part of the **Change Cipher Spec Protocol**, not the Handshake Protocol.
2. **Client Sends Finished Message**
   - ✓First message encrypted using the new algorithms, keys, and secrets.
   - ✓Confirms that key exchange and authentication were successful.
   - ✓Contains two hash values (MD5 & SHA-1)

```
MD5(master_secret || pad2 || MD5(handshake_messages ||
    Sender || master_secret || pad1))
SHA(master_secret || pad2 || SHA(handshake_messages ||
    Sender || master_secret || pad1))
```

   - ✓Sender identifies the client.

3. Server Responds

- Sends its change_cipher_spec message.
- Copies its pending CipherSpec to current.
- Sends its own finished message with similar hash verification.

4. Secure Connection Established

- Handshake is complete.
- Client and server can now exchange application-layer data securely.

TLS (Transport Layer Security)

- TLS is an IETF standardization initiative whose goal is to produce an Internet standard version of SSL.
- TLS is defined as a Proposed Internet Standard in RFC 5246 similar to SSL version3.

Differences of TLS from SSL

1. Version Number :
   - The TLS Record Format is the same as that of the SSL Record Format
   - The fields in the header have the same meanings.
   - **The one difference is in version values. For the current version of TLS, the major version is 3 and the minor version is 3.**

2. Message Authentication Code

- There are two differences between the SSLv3 and TLS MAC schemes:
  1. Actual algorithm
  2. Scope of the MAC calculation.
- TLS makes use of the HMAC algorithm defined in RFC 2104.
- HMAC is defined as

$$\text{HMAC}_K(M) = \text{H}[(K^+ \oplus \text{opad}) \| \text{H}[(K^+ \oplus \text{ipad}) \| M]]$$

where

H	=	embedded hash function (for TLS, either MD5 or SHA-1)
M	=	message input to HMAC
K^+	=	secret key padded with zeros on the left so that the result is equal to the block length of the hash code (for MD5 and SHA-1, block length = 512 bits)
ipad	=	00110110 (36 in hexadecimal) repeated 64 times (512 bits)
opad	=	01011100 (5C in hexadecimal) repeated 64 times (512 bits)

3. Alert Codes

- TLS supports all of the alert codes defined in SSLv3 with the exception of **no_certificate**.
- A number of additional codes are defined in TLS; of these, the following are always fatal.
  - **record_overflow:** A TLS record was received with a payload (ciphertext) whose length exceeds $2^{14}+2048$ bytes, or the ciphertext decrypted to a length of greater than $2^{14}+1024$ bytes.
  - **unknown_ca:** A valid certificate chain or partial chain was received, but the certificate was not accepted because the CA certificate could not be located or could not be matched with a known, trusted CA.
  - **access_denied:** A valid certificate was received, but when access control was applied, the sender decided not to proceed with the negotiation.
  - **decode_error:** A message could not be decoded, because either a field was out of its specified range or the length of the message was incorrect.

- **`protocol_version:`** The protocol version the client attempted to negotiate is recognized but not supported.
- **`insufficient_security:`** Returned instead of `handshake_failure` when a negotiation has failed specifically because the server requires ciphers more secure than those supported by the client.
- **`unsupported_extension:`** Sent by clients that receive an extended server hello containing an extension not in the corresponding client hello.
- **`internal_error:`** An internal error unrelated to the peer or the correctness of the protocol makes it impossible to continue.
- **`decrypt_error:`** A handshake cryptographic operation failed, including being unable to verify a signature, decrypt a key exchange, or validate a finished message.

The remaining alerts include the following.

- **`user_canceled:`** This handshake is being canceled for some reason unrelated to a protocol failure.
- **`no_renegotiation:`** Sent by a client in response to a hello request or by the server in response to a client hello after initial handshaking. Either of these messages would normally result in renegotiation, but this alert indicates that the sender is not able to renegotiate. This message is always a warning.

4. Cipher suits

There are several small differences between the cipher suites available under SSLv3 and under TLS:

- **Key Exchange:** TLS supports all of the key exchange techniques of SSLv3 with the exception of Fortezza.
- **Symmetric Encryption Algorithms:** TLS includes all of the symmetric encryption algorithms found in SSLv3, with the exception of Fortezza.

5. Client Certificate Types

- ✓TLS does not include rsa_ephemeral_dh, dss_ephemeral_dh, and fortezza_kea.
  - ***RSA Ephemeral DH*** *and* ***DSS Ephemeral DH*** *refer to using RSA or DSS (Digital Signature Standard) to authenticate the Diffie-Hellman parameters.*
- **✓TLS simplifies Diffie-Hellman signing by using rsa_sign and dss_sign.**
  - *Instead of having separate* ***ephemeral DH versions*** *for RSA and DSS, TLS standardized signing using* ***rsa_sign*** *(for RSA) and* ***dss_sign*** *(for DSS).*
- ✓Fortezza scheme is dropped in TLS

6. Certificate_Verify and Finished Messages

- TLS uses **MD5 and SHA-1** hashes.
- Hashes are calculated **only over handshake messages.**
- **Simpler** than SSLv3.
- Both TLS and SSLv3 use a finished message to verify the handshake.
- The **TLS Finished Message** is an essential part of the **TLS handshake**. **It is used to verify that both parties (client and server) have derived the same keys and that the handshake was not tampered with**.
- Based on:
  - ✓Master secret
  - ✓Previous handshake messages
  - ✓A label (to identify client or server).

TLS Finished Message Calculation

```
PRF(master_secret,finished_label,MD5(handshake_messages)||
     SHA-1(handshake_messages))
```

- finished_label = "client finished" (for client) or "server finished" (for server).
- PRF (Pseudo-Random Function) combines MD5 and SHA-1 for better security.

7. Padding in TLS

- ✓Allows variable padding (up to 255 bytes).
- ✓Ensures total size is a multiple of the cipher's block length.
- ✓Example: If total data size is 79 bytes, padding can be 1, 9, 17, ... 249 bytes.
- ✓Purpose: Prevents attacks based on message length analysis.